

Doc-Doctor 测试用例

文档版本：1.0
最后更新：2025-12-31
维护者：Doc-Doctor 开发团队

1. 用例编号规则

- 格式：DD-<类型>-<序号>
- 类型取值：
 - SMK：冒烟
 - UT：单元
 - FT：功能
 - IT：集成
 - REG：回归
 - AT：验收（手工）
- 自动化用例的测试名称中必须包含对应的用例 ID。

2. 测试用例矩阵（摘要）

用例ID	类型	模块/功能	优先级	自动化
DD-SMK-001	冒烟	扩展激活	P0	是
DD-SMK-002	冒烟	项目检查主流程	P0	是
DD-FT-001	功能	单文件检查（读/解析）	P0	是
DD-FT-002	功能	单文件检查（.cpp 支持）	P1	是
DD-FT-003	功能	跳转（相对路径）	P1	是
DD-FT-004	功能	跳转（绝对路径）	P1	是
DD-FT-005	功能	命令注册	P1	是
DD-UT-001~025	单元	解析、注释检查、白名单逻辑	P1	是
DD-IT-001~004	集成	项目检查	P0/P1	是
DD-IT-005~009	集成	数据库读写	P1	是
DD-REG-001~006	回归	边界/异常路径	P1	是
DD-AT-001~003	验收	UI/配置/AI 流程	P0/P1	否

3. 冒烟测试（SMK）

DD-SMK-001 扩展激活

字段	内容
测试类型	冒烟
关联模块	extension.activate
优先级	P0
前置条件	VS Code 启动扩展测试环境
测试步骤	1. 调用 <code>activate</code>
预期结果	扩展初始化不报错，返回值可为 void/undefined
自动化	是（ <code>src/test/extension.test.ts</code> ）

DD-SMK-002 项目检查主流程可执行

字段	内容
测试类型	冒烟
关联模块	projectCheck.checkAllFiles
优先级	P0
前置条件	工作区内存在 .c 文件
测试步骤	1. 调用 <code>checkAllFiles()</code>
预期结果	<code>success === true</code> 且 <code>checkedFiles > 0</code>
自动化	是（ <code>src/test/integration/projectCheck.test.ts</code> ）

4. 单元测试（UT）

DD-UT-001 解析完整注释函数

字段	内容
测试类型	单元
关联模块	fileCheck.parseFileContent
优先级	P1
前置条件	无
测试步骤	1. 调用 <code>parseFileContent(completeComment, "test.c")</code>
预期结果	<code>success=true</code> ；函数数=1；注释包含 <code>@brief/@param/@return</code>

字段	内容
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-UT-002 解析多函数文件

字段	内容
测试类型	单元
关联模块	<code>fileCheck.parseFileContent</code>
优先级	P1
测试步骤	1. 调用 <code>parseFileContent(multipleFunctions, "test.c")</code>
预期结果	识别 3 个函数，名称分别为 add/subtract/multiply
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-UT-003 过滤控制语句

字段	内容
测试类型	单元
关联模块	<code>fileCheck.parseFileContent</code>
优先级	P1
测试步骤	1. 调用 <code>parseFileContent(controlStatements, "test.c")</code>
预期结果	不识别 if/for/while/switch；仅识别 test
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-UT-004 提取函数注释块

字段	内容
测试类型	单元
关联模块	<code>fileCheck.parseFileContent</code>
优先级	P1
测试步骤	1. 调用 <code>parseFileContent(completeComment, "test.c")</code>
预期结果	注释包含 <code>/**</code> 与 <code>*/</code> 且含 <code>@brief</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-UT-005 计算函数位置

字段	内容
测试类型	单元
关联模块	fileCheck.parseFileContent
优先级	P1
测试步骤	1. 调用 <code>parseFileContent</code> (含多行注释的内容, "test.c")
预期结果	<code>lineNumber > 0</code> 且 <code>columnNumber > 0</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-UT-006 缺失 @brief 检测

字段	内容
测试类型	单元
关联模块	functionCheck.checkFunction
优先级	P1
测试步骤	1. 解析 <code>missingBrief</code> 2. 调用 <code>checkFunction</code>
预期结果	产生 1 条 <code>BRIEF_MISSING</code> 问题
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-007 缺失 @param 检测

字段	内容
测试类型	单元
关联模块	functionCheck.checkFunction
优先级	P1
测试步骤	1. 解析 <code>missingParam</code> 2. 调用 <code>checkFunction</code>
预期结果	产生 2 条 <code>PARAM_MISSING</code> 问题
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-008 缺失 @return 检测

字段	内容
测试类型	单元
关联模块	functionCheck.checkFunction

字段	内容
优先级	P1
测试步骤	1. 解析 <code>missingReturn</code> 2. 调用 <code>checkFunction</code>
预期结果	产生 1 条 <code>RETURN_MISSING</code> 问题
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-009 void 函数不要求 @return

字段	内容
测试类型	单元
关联模块	<code>functionCheck.checkFunction</code>
优先级	P1
测试步骤	1. 解析 <code>voidFunction</code> 2. 调用 <code>checkFunction</code>
预期结果	不产生 <code>RETURN_MISSING</code>
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-010 完整注释不报错

字段	内容
测试类型	单元
关联模块	<code>functionCheck.checkFunction</code>
优先级	P1
测试步骤	1. 解析 <code>completeComment</code> 2. 调用 <code>checkFunction</code>
预期结果	<code>problems.length === 0</code>
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-011 无注释函数多问题

字段	内容
测试类型	单元
关联模块	<code>functionCheck.checkFunction</code>
优先级	P1
测试步骤	1. 解析 <code>noComment</code> 2. 调用 <code>checkFunction</code>
预期结果	至少包含 BRIEF/PARAM/RETURN 问题

字段	内容
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-012 指针参数识别

字段	内容
测试类型	单元
关联模块	<code>functionCheck.checkFunction</code>
优先级	P1
测试步骤	1. 解析 <code>pointerParams</code> 2. 调用 <code>checkFunction</code>
预期结果	不产生 <code>PARAM_MISSING</code>
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-013 问题描述包含参数信息

字段	内容
测试类型	单元
关联模块	<code>functionCheck.checkFunction</code>
优先级	P1
测试步骤	1. 解析 <code>missingParam</code> 2. 调用 <code>checkFunction</code>
预期结果	<code>problemDescription</code> 包含 <code>@param</code> 和参数名
自动化	是 (<code>src/test/unit/functionCheck.test.ts</code>)

DD-UT-014 默认配置读取

字段	内容
测试类型	单元
关联模块	<code>fileWhiteList.getDocDoctorSettings</code>
优先级	P1
测试步骤	1. 调用 <code>getDocDoctorSettings()</code>
预期结果	返回默认配置结构
自动化	是 (<code>src/test/unit/fileWhiteList.test.ts</code>)

DD-UT-015 文件白名单前缀匹配

字段	内容
测试类型	单元
关联模块	fileWhiteList.isFileWhitelisted
优先级	P1
测试步骤	1. 配置 fileWhitelist 2. 调用 <code>isFilewhitelisted</code>
预期结果	命中前缀返回 true
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

DD-UT-016 空白名单不命中

字段	内容
测试类型	单元
关联模块	fileWhiteList.isFileWhitelisted
优先级	P1
测试步骤	1. 配置空白名单 2. 调用 <code>isFilewhitelisted</code>
预期结果	返回 false
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

DD-UT-017 文件级函数白名单

字段	内容
测试类型	单元
关联模块	fileWhiteList.isFunctionWhitelisted
优先级	P1
测试步骤	1. 配置函数白名单 2. 调用 <code>isFunctionwhitelisted</code>
预期结果	文件内函数匹配返回 true
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

DD-UT-018 全局函数白名单

字段	内容
测试类型	单元
关联模块	fileWhiteList.isFunctionWhitelisted

字段	内容
优先级	P1
测试步骤	1. 配置全局白名单 2. 调用 <code>isFunctionWhitelisted</code>
预期结果	全局命中返回 true
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

DD-UT-019 非白名单函数不命中

字段	内容
测试类型	单元
关联模块	<code>fileWhiteList.isFunctionWhitelisted</code>
优先级	P1
测试步骤	1. 配置函数白名单 2. 调用 <code>isFunctionWhitelisted</code>
预期结果	返回 false
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

DD-UT-020 返回值类型白名单 (void)

字段	内容
测试类型	单元
关联模块	<code>fileWhiteList.isReturnTypeWhitelisted</code>
优先级	P1
测试步骤	1. 配置 <code>returnTypeWhitelist=["void"]</code> 2. 调用 <code>isReturnTypewhitelisted</code>
预期结果	void 函数返回 true
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

DD-UT-021 返回值类型不匹配

字段	内容
测试类型	单元
关联模块	<code>fileWhiteList.isReturnTypeWhitelisted</code>
优先级	P1
测试步骤	1. 配置 <code>returnTypeWhitelist=["void"]</code> 2. 调用 <code>isReturnTypewhitelisted</code>
预期结果	非 void 返回 false

字段	内容
自动化	是 (src/test/unit/fileWhiteList.test.ts)

DD-UT-022 多关键字返回类型匹配

字段	内容
测试类型	单元
关联模块	fileWhiteList.isReturnWhitelisted
优先级	P1
测试步骤	1. 配置 returnTypeWhitelist=["unsigned long"] 2. 调用 isReturnWhitelisted
预期结果	unsigned long 返回 true
自动化	是 (src/test/unit/fileWhiteList.test.ts)

DD-UT-023 main 默认跳过

字段	内容
测试类型	单元
关联模块	fileWhiteList.shouldSkipFunction
优先级	P1
测试步骤	1. checkMainFunction=false 2. 调用 shouldSkipFunction
预期结果	返回 true
自动化	是 (src/test/unit/fileWhiteList.test.ts)

DD-UT-024 开启 main 检查

字段	内容
测试类型	单元
关联模块	fileWhiteList.shouldSkipFunction
优先级	P1
测试步骤	1. checkMainFunction=true 2. 调用 shouldSkipFunction
预期结果	返回 false
自动化	是 (src/test/unit/fileWhiteList.test.ts)

DD-UT-025 文件白名单跳过

字段	内容
测试类型	单元
关联模块	fileWhiteList.shouldSkipFunction
优先级	P1
测试步骤	1. fileWhitelist 包含 test/ 2. 调用 <code>shouldSkipFunction</code>
预期结果	返回 true
自动化	是 (<code>src/test/unit/filewhiteList.test.ts</code>)

5. 功能测试 (FT)

DD-FT-001 单文件检查 (读/解析)

字段	内容
测试类型	功能
关联模块	fileCheck.checkFile
优先级	P0
测试步骤	1. 创建 <code>test_complete.c</code> 2. 调用 <code>checkFile</code>
预期结果	成功读取并解析, 函数数=1
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-FT-002 单文件检查 (.cpp)

字段	内容
测试类型	功能
关联模块	fileCheck.checkFile
优先级	P1
测试步骤	1. 创建 <code>test.cpp</code> 2. 调用 <code>checkFile</code>
预期结果	<code>success=true</code> 且函数数=1
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-FT-003 跳转（相对路径）

字段	内容
测试类型	功能
关联模块	jumpToLocation
优先级	P1
测试步骤	1. 获取相对路径 2. 调用 <code>jumpToLocation</code>
预期结果	返回布尔值（不抛异常）
自动化	是（ <code>src/test/unit/jumpToLocation.test.ts</code> ）

DD-FT-004 跳转（绝对路径）

字段	内容
测试类型	功能
关联模块	jumpToLocation
优先级	P1
测试步骤	1. 使用绝对路径 2. 调用 <code>jumpToLocation</code>
预期结果	返回布尔值（不抛异常）
自动化	是（ <code>src/test/unit/jumpToLocation.test.ts</code> ）

DD-FT-005 命令注册

字段	内容
测试类型	功能
关联模块	extension.activate / commands
优先级	P1
测试步骤	1. 获取命令列表
预期结果	至少包含一个 <code>doc-doctor.*</code> 命令
自动化	是（ <code>src/test/extension.test.ts</code> ）

6. 集成测试（IT）

DD-IT-001 项目检查发现问题

字段	内容
测试类型	集成
关联模块	projectCheck.checkAllFiles
优先级	P0
测试步骤	1. 创建缺少注释的文件 2. 调用 <code>checkAllFiles()</code>
预期结果	<code>problems.length > 0</code>
自动化	是（ <code>src/test/integration/projectCheck.test.ts</code> ）

DD-IT-002 项目检查统计文件数

字段	内容
测试类型	集成
关联模块	projectCheck.checkAllFiles
优先级	P1
测试步骤	1. 调用 <code>checkAllFiles()</code>
预期结果	<code>totalFiles >= testFilesList.length</code> 且 <code>checkedFiles <= totalFiles</code>
自动化	是（ <code>src/test/integration/projectCheck.test.ts</code> ）

DD-IT-003 取消检查流程

字段	内容
测试类型	集成
关联模块	projectCheck.checkAllFiles
优先级	P1
测试步骤	1. 创建 <code>CancellationTokenSource</code> 并立即取消 2. 调用 <code>checkAllFiles</code>
预期结果	<code>success=false</code> 且 <code>errorMessage</code> 包含“取消”
自动化	是（ <code>src/test/integration/projectCheck.test.ts</code> ）

DD-IT-004 语法错误文件跳过

字段	内容
测试类型	集成
关联模块	projectCheck.checkAllFiles
优先级	P1
前置条件	C/C++ 诊断可用（语言服务已加载）
测试步骤	1. 创建 <code>syntax_error.c</code> 2. 调用 <code>checkAllFiles()</code>
预期结果	存在 <code>SYNTAX_ERROR</code> 问题，文件被加入 <code>skippedFiles</code>
自动化	是（ <code>src/test/integration/projectCheck.test.ts</code> ）

DD-IT-005 数据库存储

字段	内容
测试类型	集成
关联模块	database.saveProblemToDB
优先级	P1
测试步骤	1. 构造 <code>ProblemInfo</code> 2. 调用 <code>saveProblemToDB</code>
预期结果	<code>success=true</code>
自动化	是（ <code>src/test/integration/database.test.ts</code> ）

DD-IT-006 数据库读取

字段	内容
测试类型	集成
关联模块	database.loadProblemsFromDB
优先级	P1
测试步骤	1. 调用 <code>loadProblemsFromDB</code>
预期结果	<code>success=true</code> 且 <code>problems</code> 为数组
自动化	是（ <code>src/test/integration/database.test.ts</code> ）

DD-IT-007 数据库状态更新

字段	内容
测试类型	集成
关联模块	database.updateProblemStatusInDB
优先级	P1
测试步骤	1. 保存问题 2. 更新状态为 IGNORED
预期结果	返回布尔值
自动化	是 (src/test/integration/database.test.ts)

DD-IT-008 数据库清空

字段	内容
测试类型	集成
关联模块	database.clearAllProblems
优先级	P1
测试步骤	1. 调用 <code>clearAllProblems()</code>
预期结果	返回布尔值
自动化	是 (src/test/integration/database.test.ts)

DD-IT-009 Mock 模式可用

字段	内容
测试类型	集成
关联模块	database.saveProblemToDB
优先级	P1
测试步骤	1. 模拟 DLL 不可用场景 2. 调用 <code>saveProblemToDB</code>
预期结果	返回 success=true (模拟模式)
自动化	是 (src/test/integration/database.test.ts)

7. 回归测试（REG）

DD-REG-001 不支持文件类型拒绝

字段	内容
测试类型	回归
关联模块	fileCheck.parseFileContent
优先级	P1
测试步骤	1. 调用 <code>parseFileContent(..., "test.txt")</code>
预期结果	<code>success=false</code> 且 <code>errorCode=UNSUPPORTED_FILE_TYPE</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-REG-002 空文件解析

字段	内容
测试类型	回归
关联模块	fileCheck.parseFileContent
优先级	P1
测试步骤	1. 调用 <code>parseFileContent(emptyFile, "test.c")</code>
预期结果	<code>functions.length === 0</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-REG-003 注释-only 文件解析

字段	内容
测试类型	回归
关联模块	fileCheck.parseFileContent
优先级	P1
测试步骤	1. 调用 <code>parseFileContent(onlyComments, "test.c")</code>
预期结果	<code>functions.length === 0</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-REG-004 文件读取异常

字段	内容
测试类型	回归
关联模块	fileCheck.checkFile
优先级	P1
测试步骤	1. 传入不存在的 URI
预期结果	<code>success=false</code> 且 <code>errorCode=READ_ERROR</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-REG-005 checkFile 拒绝非 C/C++

字段	内容
测试类型	回归
关联模块	fileCheck.checkFile
优先级	P1
测试步骤	1. 传入 <code>.txt</code> 文件
预期结果	<code>success=false</code> 且 <code>errorCode=UNSUPPORTED_FILE_TYPE</code>
自动化	是 (<code>src/test/unit/fileCheck.test.ts</code>)

DD-REG-006 跳转不存在文件

字段	内容
测试类型	回归
关联模块	jumpToLocation
优先级	P1
测试步骤	1. 传入不存在的文件路径
预期结果	返回 <code>false</code>
自动化	是 (<code>src/test/unit/jumpToLocation.test.ts</code>)

8. 验收测试（AT）

DD-AT-001 单文件检查 UI 流程

字段	内容
测试类型	验收
关联模块	Webview + pickAndCheckFile
优先级	P0
前置条件	工作区存在 C/C++ 文件
测试步骤	1. 打开侧边栏 2. 点击“检查单个文件” 3. 选择文件
预期结果	问题列表更新，日志区提示完成
自动化	否

DD-AT-002 设置白名单并验证生效

字段	内容
测试类型	验收
关联模块	Settings + projectCheck
优先级	P1
测试步骤	1. 在设置页添加文件白名单 2. 保存设置 3. 运行项目检查
预期结果	白名单文件被跳过，问题数减少
自动化	否

DD-AT-003 AI 修复注释流程

字段	内容
测试类型	验收
关联模块	AI Refactor
优先级	P1
前置条件	配置 AI endpoint 与 apiKey
测试步骤	1. 点击“AI 修复注释” 2. 预览并应用
预期结果	注释写回源码并保存
自动化	否